Hypothesis generation and updating in large language models

Huadong Xiong
School of Psychological and Brain Sciences
Georgia Tech

Abstract

Large language models (LLMs) increasingly help people solve problems, from debugging code to repairing machinery. This process requires generating plausible hypotheses from partial descriptions, then updating them as more information arrives. Yet how LLMs perform this form of inference, and how close it is to optimal, remains unclear. We study this question in the number game, a controlled setting in which a learner infers the hypothesis supported by a few positive integers, such as $\{16, 8, 2, 64\}$: a rule like powers of 2 or an interval like numbers near 20. We measure the posterior over hypotheses using three complementary probes: posterior prediction, hypothesis evaluation, and hypothesis generation. We then compare LLM behavior with an optimal Bayesian model and human behavior, and test whether the same posterior is expressed across probes. LLMs are often well described by a two-parameter Bayesian fit, but with systematic offsets: by default they show a strong-sampling assumption that creates an implicit Occam's razor, favoring narrower hypotheses, while thinking mode shifts them toward greater prior reliance. We also find a robust evaluation–generation gap: LLMs select more correct hypotheses during hypothesis evaluation but generate simpler, more rule-like hypotheses. Finally, this Bayesian-with-bias pattern does not extrapolate. Models can behave as if they hold rule-like hypotheses over observed examples, yet generalize poorly to parts of the hypothesis domain not covered by those examples. Our results highlight a limitation of LLMs as general problem solvers, especially for scientific inference, where hypotheses must go beyond the data.

1 Introduction

Imagine fine-tuning a large language model (LLM) for a downstream task. You vibe-code a pipeline, the training loss drops, and you are happy. But on test generations, nothing improves. You quickly form plausible hypotheses: too many epochs, a wrong mask, a train/eval mismatch. After a night of tests, you narrow the possibilities and blame low-quality datasets collected by your colleagues.

Humans are naturally good scientists: we form hypotheses and test them. We adapt by building internal models of the world and using them for prediction (von Helmholtz, 1878). Children see only a few dogs yet extend the word "dog" to fluffy animals with four legs, but not the word "mom" in the same way (Clark, 1973; MacNamara, 1972; Rescorla, 1980). Mendeleev inferred periodic structure from limited, noisy measurements, and that hypothesis generalized to new elements. Sparse observations often underdetermine many explanations, yet humans can generate useful hypotheses and update them with data (Tenenbaum et al., 2011; Lake et al., 2015). This few-shot ability supports both everyday problem solving and scientific progress.

In the imminent agentic future, Mr. Ralpheseeks may simply delegate: "/ralph-loop fix this, make no mistake." As that future takes shape, from agentic coding to science (Battleday and Gershman, 2024; Cornelio et al., 2023; Novikov et al., 2025), we need to understand how current pretrained LLMs

Preprint.

generate and update hypotheses, where they fail, and what those limits imply. Recent surveys cover the broader automation landscape (Wei et al., 2025; Zheng et al., 2025).

In this paper, we investigate how pretrained LLMs form and update hypotheses in a classic controlled setting: the number game (Tenenbaum, 1999b,a). A learner observes a few positive integer examples and infers candidate hypotheses about the rule or interval that governs them. After seeing $\{16, 4, 8\}$, for example, one might consider rule-like hypotheses such as powers of 2 or even numbers, as well as interval-like hypotheses such as numbers from 1 to 20. Because the examples are sparse, they underdetermine the hypothesis and expose the learner's inductive biases. The number game's well-defined integer domain and intuitive hypothesis spaces let us ask how an intelligent system generates and updates hypotheses from a handful of examples, how those hypotheses change as examples arrive, and how consistent they remain across measurements.

We find five main results. First, LLM predictions are well described by a simple Bayesian fit (Fig. 2): models lie near, but not exactly at, the optimal Bayesian reference, and additional examples move their fitted prior–likelihood balance toward that reference. Second, by default, LLMs treat examples as if they were drawn from the same target hypothesis, producing an Occam's-razor-like bias toward narrower hypotheses; prompting them to treat examples as more incidental, or enabling thinking, reduces this bias. Third, the three measurements of the posterior are not interchangeable: posterior prediction and hypothesis generation produce closer fitted posteriors, whereas hypothesis evaluation shows a stronger preference for narrow hypotheses as more examples arrive and becomes less Bayesian-like. Fourth, hypothesis evaluation selects top hypotheses that more often contain all observed examples, whereas hypothesis generation produces simpler and more rule-like top hypotheses. Fifth, when LLMs see examples only from $\{1, \ldots, 100\}$ but are queried over $\{1, \ldots, 200\}$, they generalize poorly into the enlarged domain, suggesting that rule-like behavior over observed examples does not imply a stable latent hypothesis over the domain.

Together, these results provide a detailed measurement of LLM hypothesis generation and updating. They show that behavior depends strongly on prompts and observed examples, revealing departures from Bayesian inference and opportunities to make future models more Bayesian-like (Qiu et al., 2026). Appendix A.1 situates this framing relative to AI-for-science, Bayesian concept learning, in-context learning, and LLM probabilistic reasoning.

2 Methods

2.1 Tenenbaum's number game

The number game was introduced as a minimal setting for studying how people infer numerical hypotheses from a few positive examples (Tenenbaum, 1999b,a). Given examples such as $\{16, 4, 8\}$, a learner might infer a rule-like hypothesis such as powers of two, a broader rule such as even numbers, or a similarity-like interval around the observed examples. We use this setting because sparse positive examples underdetermine many compatible hypotheses, allowing us to measure the inductive biases that shape hypothesis generation and updating.

Formally, the hypothesis space $\mathcal{H}$ is a finite set of candidate hypotheses. Each hypothesis $h \in \mathcal{H}$ denotes a subset of a finite integer domain $D_d = \{1, \ldots, d\}$, and learning consists of inferring which subset generated the observed examples. The original number-game experiments used the finite domain $\{1, \ldots, 100\}$; we use the same domain and also test an enlarged $\{1, \ldots, 200\}$ domain. The hypothesis space combines two intuitive families: rule-like hypotheses, such as mathematical patterns, and similarity-like hypotheses, such as contiguous magnitude intervals. Following Tenenbaum (1999b,a), the prior assigns mass across these rule and interval hypotheses; construction details and prior parameters are given in Appendix A.5. The Bayesian reference and the (α, β) fits use this configured hypothesis set, and the hypothesis-evaluation lists are example-conditioned views of the same underlying rule-and-interval construction.

The likelihood is determined by the assumed sampling process for positive examples. Under strong sampling, examples are assumed to be drawn uniformly from inside the true hypothesis; under weak sampling, they are only known to be positive instances. Strong sampling yields the *size principle*: observing several examples inside a small hypothesis is more informative than observing the same examples inside a broad hypothesis. If X is the observed examples and $|h|$ is the number of domain elements in h, the strong-sampling likelihood is proportional to

$$p(X \mid h) \propto |h|^{-|X|} \quad \text{if } X \subseteq h, \tag{1}$$

and zero otherwise. Thus four examples consistent with powers of two are much more likely under that compact rule than under a broad hypothesis such as even numbers. This is the Occam's-razor effect implicit in strong sampling: among hypotheses that explain the examples, smaller compatible hypotheses receive more posterior support.

The key Bayesian prediction is hypothesis averaging: a posterior probability mass function over hypotheses is turned into integer-level predictions by averaging over hypotheses,

$$p(y \in h^\star \mid X) = \sum_{h \in \mathcal{H}} \mathbf{1}[y \in h]\, p(h \mid X). \tag{2}$$

To compare LLM behavior to this reference, we fit a two-parameter Bayesian family over hypotheses. The parameter α measures reliance on the configured prior over hypotheses: larger values preserve the prior ordering of rule-like and similarity-like hypotheses more strongly after examples are observed. The parameter β measures the strength of the sampling assumption in the likelihood: $\beta = 1$ gives the full strong-sampling size principle, whereas $\beta = 0$ removes the penalty on broad compatible hypotheses. Thus $(\alpha, \beta) = (1, 1)$ is the configured Bayesian reference: the original prior combined with the strong-sampling likelihood. The full parameterization and fitting objective are given in Appendix A.6.

2.2 Probing LLMs' posterior over hypotheses via three measurements

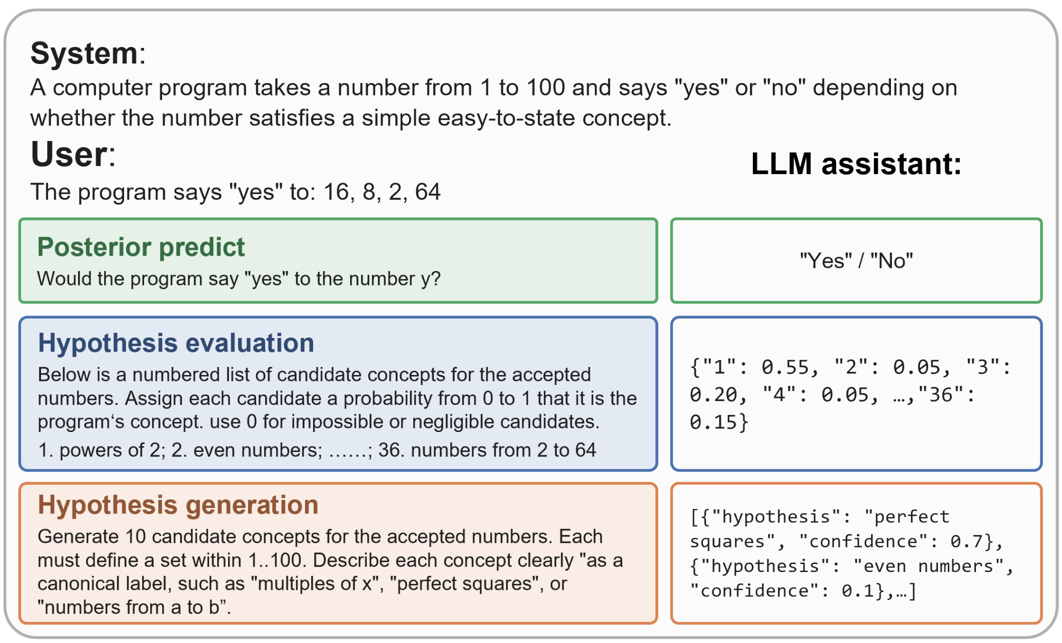

Figure 1: **Three measurements of the posterior over hypotheses in the number game.** The schematic shows how we prompt LLMs to measure their posterior over hypotheses. Posterior prediction queries the LLM with one integer in the hypothesis domain at a time and records the model-generated probability that the integer belongs to the same hypothesis as the current examples, yielding a probability mass function. Hypothesis evaluation shows a candidate list of all hypotheses used by the Bayesian model for the current examples and records the LLM's confidence in those labels. Hypothesis generation asks the LLM to generate 10 different hypotheses and associated confidences given the examples.

We use three probes to measure LLMs' posterior over hypotheses given examples X. Posterior prediction queries every integer y in the hypothesis domain D_d and records $q_m^{(d)}(y \mid X)$ from a forced Yes/No target question (Appendix A.3). Hypothesis evaluation provides a compact, example-conditioned list of candidate hypotheses drawn from the same rule-and-interval construction used by the Bayesian reference and asks for the confidence assigned to each hypothesis (Appendix A.5). Hypothesis generation asks the model to propose 10 hypotheses that describe the seen examples, with an associated confidence for each hypothesis. Under a Bayesian model, these three measurements should be different readouts of one posterior $p(h \mid X)$. We compare them by projecting evaluation and generation into the same predictive space as posterior prediction. This projection lets us ask whether the measurement itself changes the hypotheses the model generates and updates, with all three measurements represented in a common space. Full candidate-list and projection details are given in Appendices A.5 and A.7.

2.3 Stimuli and models

We evaluate two number-game sources. TENENBAUM99 contains eight hand-designed stimulus sets from the classic number-game experiments, including rule-evoking sets such as $\{16, 8, 2, 64\}$ and similarity-evoking sets such as $\{16, 23, 19, 20\}$. BIGELOW16 contains 255 stimulus sets from the broader Bigelow and Piantadosi human dataset (Bigelow and Piantadosi, 2016a,b). Models see each stimulus set one example at a time, up to four examples. This produces 26 observed-example presentations per measurement for TENENBAUM99 and 636 presentations for BIGELOW16 in $d = 100$; Appendix A.4 gives the stimulus categories, and Appendix A.6 gives the task-pooling and example-count scopes used in aggregate fits.

The main non-thinking model panel contains eight pretrained LLMs: the Gemma 4 family (A4B, E4B, and E2B) (Team, 2026); the Qwen family (Qwen 3.6 A3B, Qwen 3.5 4B, and Qwen 3.5 2B) (Qwen, 2026); GPT-5.4 Mini; and Nemotron 3 Nano (NVIDIA et al., 2025). Thinking-mode comparisons use matched thinking and non-thinking runs for six of these models: all except Gemma 4 E2B and Qwen 3.5 2B. To understand LLM behavior, we fit a Bayesian model with (α, β) as free parameters. Across measurements, we project model responses into posterior predictive distributions for analysis. To test whether LLM hypotheses generalize, we also prompt models with an enlarged hypothesis domain, $\{1, \dots, 200\}$, while the examples remain in $\{1, \dots, 100\}$. Additional experimental details are given in Appendices A.2–A.8.

3 Results

3.1 A Bayesian fit describes LLM posterior prediction behavior

Following Tenenbaum's Bayesian number-game model, we ask whether LLM posterior predictions can be described by the two-parameter Bayesian family defined above. The fit measures how close the models are to the Bayesian reference and identifies which part of the Bayesian computation they approximate: the prior over hypotheses or the likelihood induced by the sampling assumption. We therefore fit (α, β) to each model's posterior predictions on the shared rule-and-interval hypothesis space, using the configured Bayesian model at $(1, 1)$ as the reference point. Unless a task-specific analysis is stated, each reported fit pools the available TENENBAUM99 and BIGELOW16 presentations for the fixed model, domain, prompt condition, and fit scope.

LLM posterior predictions remain close to the Bayesian reference, but with systematic model-specific offsets. In the default posterior-prediction setting, most full-stimulus fits fall in a compact region around $(1, 1)$ rather than far from the configured Bayesian reference (Fig. 2a). Some models have lower fitted β, indicating a weaker size-principle effect; others have higher fitted α, indicating stronger prior reliance. Human behavior is also displaced from $(1, 1)$, especially toward lower β, making the human baseline more prior-dominated than the configured reference.

We next refit the two-parameter summary separately for different numbers of in-context examples, $n = 1, 2, 3, 4$, asking how LLM behavior changes as examples arrive. As the number of observed examples increases from one to four, the LLM trajectories generally move toward $\log(\alpha/\beta) \approx 0$, indicating a fitted balance closer to the Bayesian reference (Fig. 2b). Averaged over LLMs, early predictions are more prior-weighted, but additional examples increase the relative influence of the size-principle term, as expected when each example makes narrower compatible hypotheses more diagnostic. Humans remain more prior-dominated across numbers of in-context examples. Taken

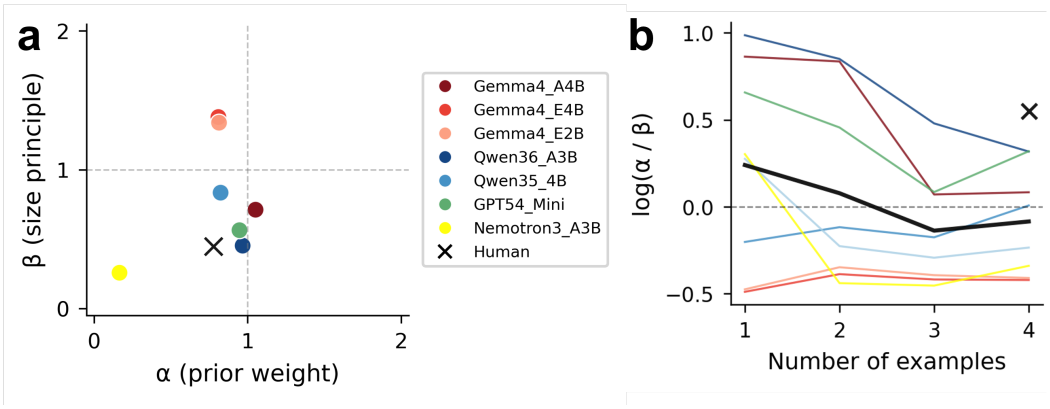

Figure 2: **Bayesian fit of LLM posterior prediction behavior.** **a**, Full-stimulus (α, β) fits for default $d = 100$ posterior prediction, with each model fit once after pooling all available full-stimulus TENENBAUM99 and BIGELOW16 presentations. Each colored point is one model; dashed lines mark the configured Bayesian reference, $(1, 1)$; the human baseline is shown as a black cross. **b**, Example-count trajectories of $\log(\alpha/\beta)$ over one to four examples, using the same task-pooling rule within each example-count scope. Each colored line is one model's mean trajectory across complete-prefix stimulus rows, the thick black line is the mean across LLM models, and the black cross marks the human endpoint. The dashed zero line marks the Bayesian balance point where fitted prior and size-principle weights are equal; positive values indicate stronger fitted prior influence relative to fitted size-principle influence.

together, these results reveal a Bayesian-with-bias pattern: LLMs are often well described by a Bayesian fit, but with modest systematic offsets from the optimal $\alpha = \beta = 1$ reference.

3.2 LLMs show strong-sampling behavior

The Bayesian likelihood depends critically on whether examples are assumed to be drawn from the hidden hypothesis or merely observed as positive instances without a sampling process. The former is strong sampling; the latter is weak sampling. Strong sampling induces a size principle that favors simpler hypotheses with smaller support. We therefore ask which behavior LLMs show by varying the sampling story in the prompt and by optionally showing the compact candidate-hypothesis list before prediction. We compare the Default Prompt, Strong Prompt, Weak Prompt, and Explicit Prompt; full prompt definitions are given in Appendix A.2.

The Default Prompt and Strong Prompt produce similar full-stimulus (α, β) summaries, indicating that LLM behavior closely matches the strong-sampling hypothesis even when that sampling process is not stated explicitly (Fig. 3). Weak prompts shift the fit toward higher α and lower β, consistent with the Bayesian intuition that weakening the likelihood makes the prior more dominant. The Explicit Prompt also increases α without clearly improving posterior Kullback–Leibler divergence (KL), so showing the candidate list does not by itself recover the Bayesian posterior. Sampling assumptions also separate prompt conditions by KL from the Bayesian reference: the Strong Prompt has the lowest KL, the Default Prompt is next, and the Weak Prompt is largest, as expected when the reference model itself uses a strong-sampling likelihood (Fig. 3).

3.3 Thinking reduces the size principle

In paired thinking rows, enabling thinking tends to increase α and decrease β, shifting many conditions toward stronger prior reliance and weaker size-principle behavior. Its effect on KL is mixed rather than uniformly beneficial, so thinking changes posterior shape without simply making the model more Bayesian. This is a behavioral observation about the readout, not evidence that the model internally adopts the Strong or Weak sampling assumption.

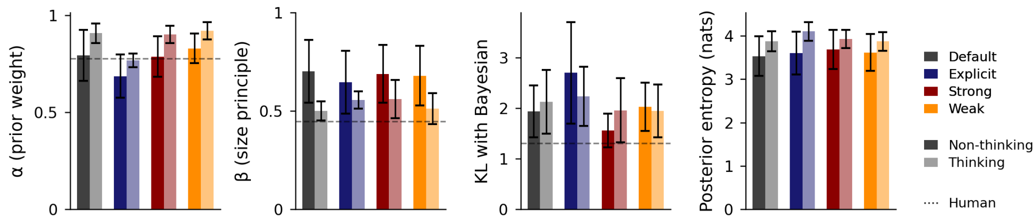

Figure 3: **Bayesian fits quantify how prompt sampling assumptions, explicit candidate lists, and thinking affect LLM posterior prediction behavior.** Bars summarize model-averaged posterior-prediction conditions for the Default Prompt, Strong Prompt, Weak Prompt, and Explicit Prompt. Within each prompt condition, dark bars show non-thinking model rows and lighter bars show thinking model rows. The left two panels report full-stimulus α and β fits, with dashed lines marking the configured Bayesian value of 1; the right two panels report KL divergence from Bayesian posterior predictions and posterior entropy. Error bars show 95% confidence intervals across model-level condition means.

3.4 Posterior measurements reveal behavioral gaps

For a Bayesian model, posterior prediction, hypothesis evaluation, and hypothesis generation should produce mutually consistent posterior predictive behavior. We ask whether LLMs expose such an internal Bayesian model when they generate and update hypotheses. We therefore project evaluation and generation confidences into that shared space, using the rule and interval supports described in Appendix A.5 and the projection procedure in Appendix A.7, and then fit (α, β) separately to each measurement. This fit lets us compare fitted prior weight, fitted size-principle strength, KL from the Bayesian posterior, and predictive entropy under the three measurements.

The three measurements do not agree (Fig. 4). Posterior prediction and hypothesis generation are closer to each other in fitted (α, β), whereas hypothesis evaluation separates from both: it has smaller α, larger β, and a larger KL from the Bayesian posterior, indicating a stronger fitted preference for narrower compatible hypotheses and greater distortion after projection. Thinking effects also differ by measurement rather than following one global direction. Thus, unlike an optimal Bayesian model, LLMs do not expose a single posterior that can be read out equivalently by prediction, evaluation, and generation. The gap across measurements is itself evidence that LLM behavior is not adequately described as optimal Bayesian inference over one posterior.

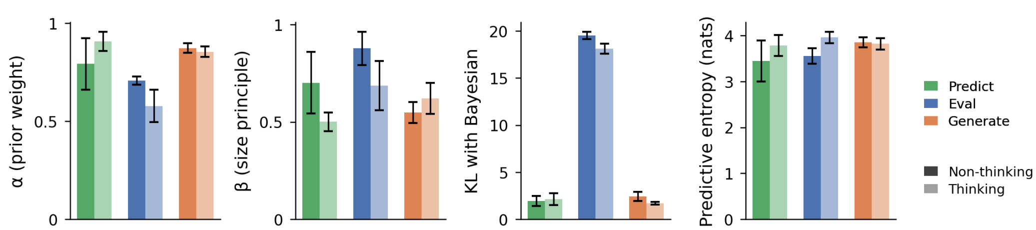

Figure 4: **Large language models show different behavior under three measurements of the posterior.** Bars compare posterior prediction (Predict), hypothesis evaluation (Eval), and hypothesis generation (Generate) after projecting each measurement into the same posterior predictive space. Within each measurement, dark bars show non-thinking model rows and lighter bars show thinking model rows. The panels report full-stimulus α and β fits, KL divergence from the Bayesian posterior, and predictive entropy; error bars show 95% confidence intervals across models.

3.5 Evaluation chooses more accurate hypotheses; generation favors simpler ones

Having shown that different measurements of the posterior over hypotheses lead to different behavior, we ask what inductive biases they reveal when models select a hypothesis. We compare the maximum a posteriori (MAP) estimator in hypothesis evaluation and generation: the top-1 hypothesis assigned the largest weight. We measure how simple that top-1 hypothesis is by its support fraction over the full hypothesis domain. For example, even numbers have support fraction 0.5, so this metric captures how much of the domain the top hypothesis covers. We also measure how accurately the top hypothesis describes the observed in-context examples.

Across the eight LLMs in non-thinking mode, evaluation selects top-1 hypotheses that more accurately describe the observed examples, whereas generation prefers narrower hypotheses (Fig. 5a,b,c). This pattern suggests an accuracy–simplicity trade-off. Broad hypotheses are more likely to include the examples but are less precise; narrow hypotheses are simpler and more informative, but may be less accurate.

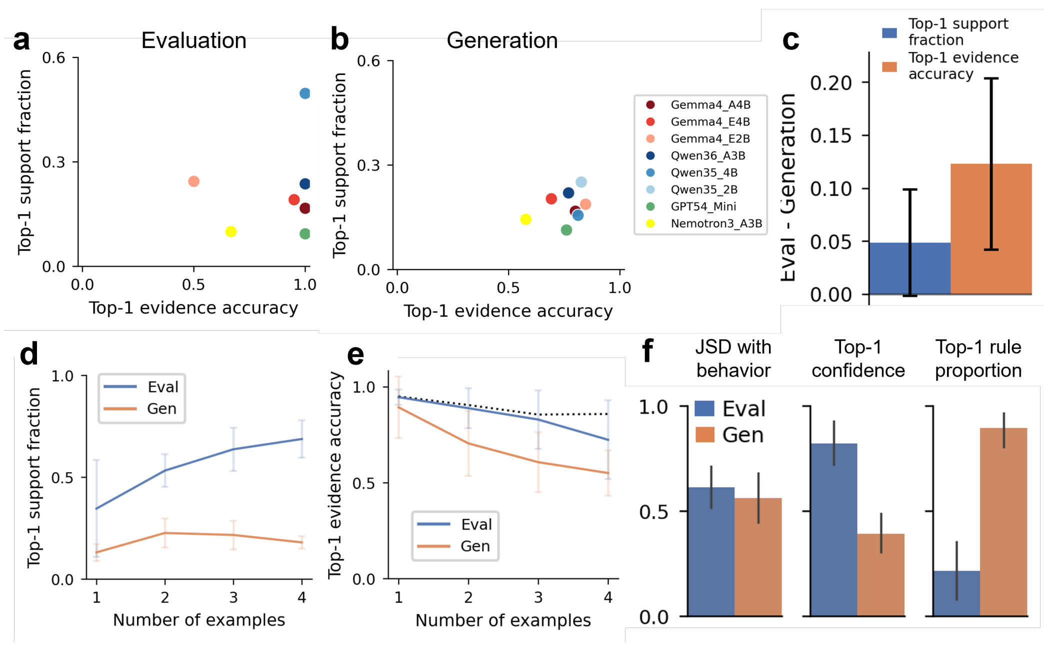

Figure 5: **Hypothesis evaluation and generation show an accuracy–simplicity trade-off in their MAP hypothesis estimators. a,b**, Top-1 example consistency versus support fraction for hypothesis evaluation and generation on TENENBAUM99 default $d = 100$ rows. **c**, Paired Eval-minus-Generation gaps for example consistency and support fraction. **d,e**, Trajectories of top-1 support fraction and example consistency as the number of examples increases; the dotted black line in **e** marks the human-description reference. **f**, Paired Eval-vs-Generation summaries for projected Jensen–Shannon distance from posterior prediction, top-1 confidence, and top-1 rule proportion. Error bars show 95% confidence intervals across models.

We then study how this inductive bias changes as the number of examples increases. Evaluation shifts toward broader supported regions as more examples arrive, whereas generation remains narrow (Fig. 5d). The two measurements start with similar accuracy in explaining the observed examples, but evaluation becomes more accurate as examples accumulate. This suggests that generation favors simpler hypotheses at the cost of accuracy, reflecting a strong implicit Occam's-razor tendency.

We also find that the MAP estimators from evaluation and generation have similar Jensen–Shannon distance from posterior prediction after projection into predictive space, but evaluation assigns higher top-1 confidence, whereas generation produces more rule-form top hypotheses (Fig. 5f). This evaluation–generation gap is therefore the clearest structural non-Bayesian signature in LLM behavior.

3.6 Poor extrapolation to the domain of unobserved examples

In scientific reasoning, a useful hypothesis must organize cases beyond the examples that produced it. We therefore ask whether LLMs can generalize when they see examples in $\{1, \ldots, 100\}$ but the hypothesis domain is expanded to $\{1, \ldots, 200\}$.

A Bayesian model should preserve its posterior shape on $1..100$ after renormalization and add structured mass in the unseen window $101..200$ only when the hypothesis is rule-based, since interval-based hypotheses with no examples in $101..200$ should not extend into that range. Rule-derived examples should therefore support extrapolation into the new range, whereas interval-derived examples should increasingly favor hypotheses contained within the original range.

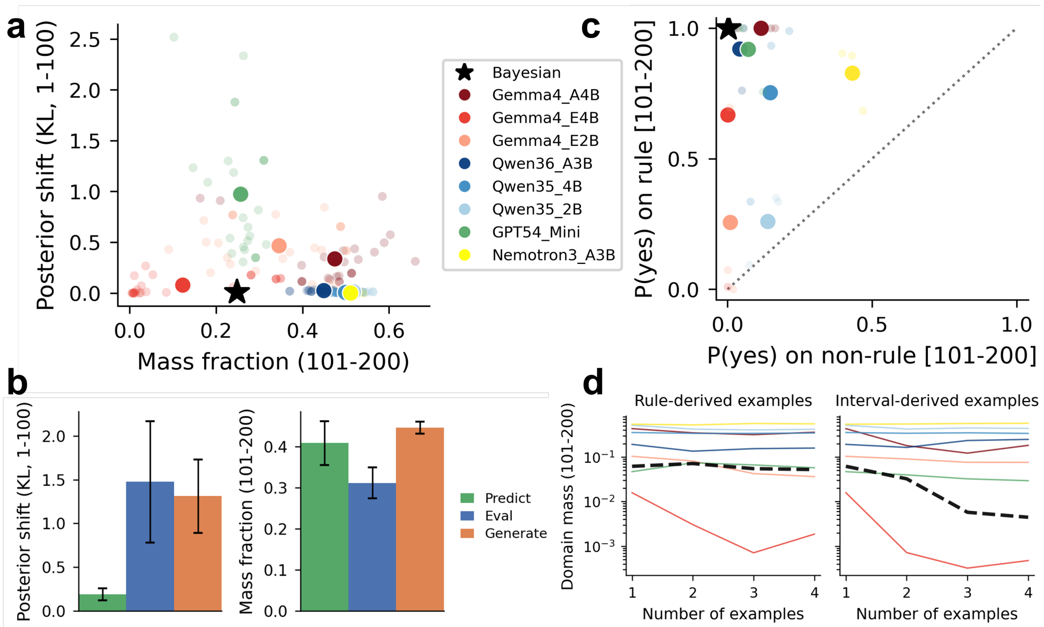

Figure 6: **LLM hypotheses fail to generalize to the unobserved domain.** **a**, Larger-domain posterior prediction, comparing mass assigned to $101..200$ with KL divergence between the original $d = 100$ posterior and the renormalized $d = 200$ posterior on $1..100$. Transparent points show stimuli; larger points show model averages. **b**, The same unobserved-domain comparison across posterior prediction, hypothesis evaluation, and hypothesis generation. **c**, Posterior-prediction discrimination between rule-based and non-rule-based examples in the unseen domain. **d**, Unobserved-domain mass over example length in the $d = 200$ condition, split by rule-derived and interval-derived stimulus sets. Error bars show 95% confidence intervals across models; the Bayesian model is dashed black.

We next test whether this apparent in-domain hypothesis updating supports extrapolation. Models receive examples drawn from $1..100$, but the query domain is expanded to $1..200$. A learner that carries the same hypothesis forward should preserve the posterior's renormalized shape on $1..100$ while assigning structured mass to $101..200$. The Bayesian reference does so, but most LLMs diverge: some assign substantial probability mass to unseen numbers while failing to preserve the original in-domain shape (Fig. 6a). Across the three measurements, posterior prediction is closest to the Bayesian reference under this domain change, whereas hypothesis evaluation and hypothesis generation are more distorted after projection (Fig. 6b).

The more diagnostic question is whether probability assigned to $101..200$ is hypothesis-selective. A rule hypothesis should extrapolate into the expanded range, whereas an interval hypothesis supported only by examples in $1..100$ should remain bounded. The Bayesian reference shows this distinction, but LLMs generally do not (Fig. 6c,d). High rule-target probabilities are often accompanied by elevated non-rule probabilities, or both probabilities are suppressed, weakening the behavioral distinction between principled extrapolation and broad leakage (Fig. 6c). The trajectory analysis gives the same conclusion over example length: Bayesian extension mass remains stable for rule-derived examples and drops for interval-derived examples, whereas most LLM trajectories are flatter and less separated across the two stimulus types (Fig. 6d). Thus, the Bayesian-like in-domain behavior observed in earlier sections does not translate into robust hypothesis-guided generalization beyond the observed examples.

4 Discussion and limitations

Two patterns summarize the results. Within a fixed measurement and query domain, LLM behavior is often well described by the two-parameter Bayesian fit, with model-specific offsets in α and β that shift under prompt sampling assumptions and thinking conditions. Across measurements and domains, however, the same models fail posterior coherence: prediction, evaluation, and generation project to different regions of the (α, β) plane (Section 3.4), and rule-like behavior over seen examples does not become structured extrapolation over a larger domain (Section 3.6). Under one posterior $p(h \mid X)$, these readouts should agree. The evaluation–generation gap and the larger-domain failure therefore point to the same limitation: current LLMs produce hypothesis-shaped outputs and partial Bayesian-like updating, but not one stable posterior expressed across probes.

The evaluation–generation gap also clarifies why LLM-as-judge systems can work well. Judging supplies a candidate hypothesis, answer, or trajectory; generation requires search and commitment. In our task, supplied candidates are more consistent with the observed examples, whereas free generation favors narrower, more rule-like hypotheses even when they fit less well. Evaluation may therefore succeed by removing part of the search problem, and bootstrapping from generated material may work when a stronger evaluative channel filters imperfect candidates.

Several limitations qualify these conclusions. The number game is deliberately small: its one-dimensional integer domain and two intuitive hypothesis families make the Bayesian reference enumerable, but restrict the claim to inductive tasks of this form. The stimulus pool is also narrow, with hypothesis generation analyzed most directly on TENENBAUM99. The (α, β) geometry and cross-measurement projections are defined relative to the common rule-and-interval reference in Appendix A.5, so the reported separation is a relative comparison under that reference rather than a reference-independent measure of incoherence. The evaluation–generation comparison is asymmetric by design, because evaluation supplies candidates whereas generation requires search; equalizing the prompts would collapse the probes. The larger-domain result could also reflect a domain-conditioned posterior that is coherent within each prompted domain but not transported across domains. Finally, the main-text analysis uses a single cached run per model with a single seed, so error bars summarize variation across models or matched rows rather than repeated stochastic decoding. The next step is to test whether the same pattern recurs in adjacent inductive tasks such as sequence extrapolation, symbolic rule learning, and function induction.

5 Conclusion

We used a two-parameter Bayesian fit to study hypothesis generation and updating in LLMs. The results show that LLM behavior is not simply noisy Bayesian inference. By default, LLMs apply a stronger size principle than the Bayesian model, behaving as if examples were drawn under strong sampling even when the prompt does not require it; weak-sampling and thinking conditions shift this offset but do not eliminate it. Hypothesis evaluation and hypothesis generation, which should agree under a single posterior $p(h \mid X)$, project to different regions of the (α, β) plane: generation systematically favors narrower, rule-form hypotheses that agree less well with the observed examples than the broader hypotheses selected by evaluation. When the query domain extends beyond the observed examples, the rule-vs-interval distinction that organizes within-domain behavior largely collapses, indicating that the hypothesis structure visible inside the seen examples is not carried forward as a stable posterior. These divergences are properties of how current LLMs assemble hypothesis-shaped outputs, not residual noise around a Bayesian center. LLMs should therefore be judged not by whether they can state a plausible explanation under any single probe, but by whether the same explanation remains coherent when it is predicted, evaluated, generated, and extended beyond the observed examples; on this criterion, current LLMs systematically fall short.

References

Ahn, K., Cheng, X., Daneshmand, H., and Sra, S. (2023). Transformers learn to implement preconditioned gradient descent for in-context learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 45614–45650.

Bai, Y., Chen, F., Wang, H., Xiong, C., and Mei, S. (2023). Transformers as Statisticians: Provable In-Context Learning with In-Context Algorithm Selection.

Battleday, R. M. and Gershman, S. J. (2024). Artificial intelligence for science: The easy and hard problems. arXiv:2408.14508 [cs].

Bazigaran, A. and Sohn, H. (2025). Concept Generalization in Humans and Large Language Models: Insights from the Number Game. arXiv:2512.20162 [cs].

Bigelow, E. and Piantadosi, S. T. (2016a). A large dataset of generalization patterns in the number game. *Journal of Open Psychology Data*, 4(1):e4–e4.

Bigelow, E. J. and Piantadosi, S. T. (2016b). Inferring priors in compositional cognitive models. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 38.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Chen, Y. and Wang, X. (2022). Transformers as Meta-Learners for Implicit Neural Representations. arXiv:2208.02801 [cs].

Cheng, X., Chen, Y., and Sra, S. (2024). Transformers Implement Functional Gradient Descent to Learn Non-Linear Functions In Context. arXiv:2312.06528 [cs].

Clark, E. V. (1973). What's in a word? On the child's acquisition of semantics in his first language. In *Cognitive development and acquisition of language*, pages 65–110. Elsevier.

Cornelio, C., Dash, S., Austel, V., Josephson, T. R., Goncalves, J., Clarkson, K. L., Megiddo, N., El Khadir, B., and Horesh, L. (2023). Combining data and theory for derivable scientific discovery with AI-Descartes. *Nature Communications*, 14(1):1777.

Dherin, B., Munn, M., Mazzawi, H., Wunder, M., and Gonzalvo, J. (2025). Learning without training: The implicit dynamics of in-context learning. arXiv:2507.16003 [cs].

Garg, S., Tsipras, D., Liang, P. S., and Valiant, G. (2022). What Can Transformers Learn In-Context? A Case Study of Simple Function Classes. *Advances in Neural Information Processing Systems*, 35:30583–30598.

Lakatos, I. (2014). Falsification and the methodology of scientific research programmes. In *Philosophy, science, and history*, pages 89–94. Routledge.

Lake, B. M., Salakhutdinov, R., and Tenenbaum, J. B. (2015). Human-level concept learning through probabilistic program induction. *Science*, 350(6266):1332–1338.

MacNamara, J. (1972). Cognitive basis of language learning in infants. *Psychological Review*, 79(1):1–13.

Müller, S., Hollmann, N., Arango, S. P., Grabocka, J., and Hutter, F. (2021). Transformers Can Do Bayesian Inference.

Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. (2025). AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs].

NVIDIA, Blakeman, A., Grattafiori, A., Basant, A., Gupta, A., Khattar, A., Renduchintala, A., Vavre, A., Shukla, A., Bercovich, A., Ficek, A., Shaposhnikov, A., Kondratenko, A., Bukharin, A., Milesi, A., Taghibakhshi, A., Liu, A., Barton, A., Mahabaleshwarkar, A. S., Klein, A., Zuker, A., Geifman, A., Shen, A., Bhiwandiwalla, A., Tao, A., Agrusa, A., Verma, A., Guan, A., Mandarwal, A., Mehta, A., Aithal, A., Poojary, A., Ahamed, A., Mishra, A., Thekkumpate, A. K., Dattagupta, A., Zhu, B., Sadeghi, B., Simkin, B., Lanir, B., Schifferer, B., Nushi, B., Kartal, B., Rouhani, B. D., Ginsburg, B., Norick, B., Soubasis, B., Kisacanin, B., Yu, B., Catanzaro, B., Mundo, C. d., Hwang, C., Wang, C., Hsieh, C.-P., Zhang, C., Yu, C., Mungekar, C., Patel, C., Alexiuk, C., Parisien, C., Neale, C., Meurillon, C., Mosk-Aoyama, D., Su, D., Corneil, D., Afrimi, D., Lo, D., Rohrer, D., Serebrenik, D., Gitman, D., Levy, D., Stosic, D., Mosallanezhad, D., Narayanan, D., Nathawani, D., Rekesh, D., Yared, D., Kakwani, D., Ahn, D., Riach, D., Stosic, D., Minasyan, E., Lin, E., Long, E., Long, E. P., Segal, E., Lantz, E., Evans, E., Ning, E., Chung, E., Harper, E., Tramel, E., Galinkin, E., Pounds, E., Briones, E., Bakhturina, E., Tsykunov, E., Ladhak, F., Wang, F., Jia, F., Soares, F., Chen, F., Galko, F., Sun, F., Siino, F., Agam, G. H., Ajjanagadde, G., Bhatt, G., Prasad, G., Armstrong, G., Shen, G., Batmaz, G., Nalbandyan, G., Qian, H., Sharma, H., Ross, H., Ngo, H., Hum, H., Sahota, H., Wang, H., Soni, H., Upadhyay, H., Mao, H., Nguyen, H. C., Nguyen, H. Q., Cunningham, I., Galil, I., Shahaf, I., Gitman, I., Loshchilov, I., Schen, I., Levy, I., Moshkov, I., Golan, I., Putterman, I., Kautz, J., Scowcroft, J. P., Casper, J., Mitra, J., Glick, J., Chen, J., Oliver, J., Zhang, J., Zeng, J., Lou, J., Zhang, J., Choi, J., Huang, J., Conway, J., Guman, J., Kamalu, J., Greco, J., Cohen, J., Jennings, J., Daw, J., Vialard, J. V., Yi, J., Parmar, J., Xu, K., Zhu, K., Briski, K., Cheung, K., Luna, K., Wyss, K., Santhanam, K., Shih, K., Kong, K., Bhardwaj, K., Shankar, K., Puvvada, K. C., Pawelec, K., Anik, K., McAfee, L., Sleiman, L., Derczynski, L., Ding, L., Wei, L., Liebenwein, L., Vega, L., Grover, M., Segbroeck, M. V., Melo, M. R. d., Nazemi, M., Sreedhar, M. N., Kilaru, M., Ashkenazi, M., Romeijn, M., Chochowski, M., Cai, M., Kliegl, M., Moosaei, M., Kulka, M., Novikov, M., Samadi, M., Corpuz, M., Wang, M., Price, M., Andersch, M., Boone, M., Evans, M., Martinez, M., Khona, M., Chrzanowski, M., Lee, M., Dabbah, M., Shoeybi, M., Patwary, M., Mulepati, N., Nabwani, N., Hereth, N., Assaf, N., Habibi, N., Zmora, N., Haber, N., Sessions, N., Bhatia, N., Jukar, N., Pope, N., Ludwig, N., Tajbakhsh, N., Ailon, N., Juluru, N., Sharma, N., Hrinchuk, O., Kuchaiev, O., Delalleau, O., Olabiyi, O., Argov, O. U., Puny, O., Tropp, O., Xie, O., Chadha, P., Shamis, P., Gibbons, P., Molchanov, P., Morkisz, P., Dykas, P., Jin, P., Xu, P., Januszewski, P., Thombre, P. P., Varshney, P., Gundecha, P., Tredak, P., Miao, Q., Wan, Q., Mahabadi, R. K., Garg, R., El-Yaniv, R., Zilberstein, R., Shafipour, R., Harang, R., Izzo, R., Shahbazyan, R., Garg, R., Borkar, R., Gala, R., Islam, R., Hesse, R., Waleffe, R., Watve, R., Koren, R., Zhang, R., Hewett, R., Hewett, R. J., Prenger, R., Timbrook, R., Mahdavi, S., Modi, S., Kriman, S., Lim, S., Kariyappa, S., Satheesh, S., Kaji, S., Pasumarthi, S., Muralidharan, S., Narentharen, S., Narenthiran, S., Bak, S., Kashirsky, S., Poulos, S., Mor, S., Ramasamy, S., Acharya, S., Ghosh, S., Sreenivas, S. T., Thomas, S., Fan, S., Gopal, S., Prabhumoye, S., Pachori, S., Toshniwal, S., Ding, S., Singh, S., Sun, S., Ithape, S., Majumdar, S., Singhal, S., Sergienko, S., Alborghetti, S., Ge, S., Devare, S. D., Barua, S. K., Panguluri, S., Gupta, S., Priyadarshi, S., Akter, S. N., Bui, T., Ene, T.-D., Kong, T., Do, T., Blankevoort, T., Moon, T., Balough, T., Asida, T., Natan, T. B., Ronen, T., Konuk, T., Vashishth, T., Karpas, U., De, U., Noorozi, V., Noroozi, V., Srinivasan, V., Elango, V., Cui, V., Korthikanti, V., Rao, V., Kurin, V., Lavrukhin, V., Anisimov, V., Jiang, W., Ahmad, W. U., Du, W., Ping, W., Zhou, W., Jennings, W., Zhang, W., Prazuch, W., Ren, X., Karnati, Y., Choi, Y., Meyer, Y., Wu, Y.-F., Zhang, Y., Qin, Y., Lin, Y., Geifman, Y., Fu, Y., Subara, Y., Suhara, Y., Gao, Y., Moshe, Z., Dong, Z., Zhu, Z., Liu, Z., Chen, Z., and Yan, Z. (2025). NVIDIA Nemotron 3: Efficient and Open Intelligence. arXiv:2512.20856 [cs].

Padmanabhan, S., Misra, K., Mahowald, K., and Choi, E. (2025). On Language Models' Sensitivity to Suspicious Coincidences. arXiv:2504.09387 [cs].

Popper, K. (2005). *The logic of scientific discovery*. Routledge.

Qiu, L., Sha, F., Allen, K., Kim, Y., Linzen, T., and van Steenkiste, S. (2026). Bayesian teaching enables probabilistic reasoning in large language models. *Nature Communications*, 17(1):1238.

Qwen, T. (2026). Qwen3.5: Towards Native Multimodal Agents. original-date: 2025-09-11T05:32:39Z.

Rescorla, L. A. (1980). Overextension in early language development. *Journal of Child Language*, 7(2):321–335.

Reuter, A., Rudner, T. G. J., Fortuin, V., and Rügamer, D. (2025). Can Transformers Learn Full Bayesian Inference in Context?

Team, G. (2026). Gemma 4: Byte for byte, the most capable open models.

Tenenbaum, J. (1999a). Rules and Similarity in Concept Learning. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press.

Tenenbaum, J. B., Kemp, C., Griffiths, T. L., and Goodman, N. D. (2011). How to Grow a Mind: Statistics, Structure, and Abstraction. *Science*, 331(6022):1279–1285.

Tenenbaum, J. B. J. B. (1999b). *A Bayesian framework for concept learning*. Thesis, Massachusetts Institute of Technology. Accepted: 2005-05-19T14:18:52Z.

von Helmholtz, H. (1878). The Facts of Perception. In Russell, K., editor, *Selected Writings of Hermann von Helmholtz*. Wesleyan University Press, Connecticut.

von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. (2023a). Transformers Learn In-Context by Gradient Descent. In *Proceedings of the 40th International Conference on Machine Learning*, pages 35151–35174. PMLR.

von Oswald, J., Niklasson, E., Schlegel, M., Kobayashi, S., Zucchet, N., Scherrer, N., Miller, N., Sandler, M., Arcas, B. A. y., Vladymyrov, M., Pascanu, R., and Sacramento, J. (2023b). Uncovering mesa-optimization algorithms in Transformers. arXiv:2309.05858 [cs].

Wei, J., Yang, Y., Zhang, X., Chen, Y., Zhuang, X., Gao, Z., Zhou, D., Wang, G., Gao, Z., Cao, J., Qiu, Z., Hu, M., Ma, C., Tang, S., He, J., Song, C., He, X., Zhang, Q., You, C., Zheng, S., Ding, N., Ouyang, W., Dong, N., Cheng, Y., Sun, S., Bai, L., and Zhou, B. (2025). From AI for Science to Agentic Science: A Survey on Autonomous Scientific Discovery. arXiv:2508.14111 [cs].

Xie, S. M., Raghunathan, A., Liang, P., and Ma, T. (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. arXiv:2111.02080 [cs].

Zhang, Y., Zhang, F., Yang, Z., and Wang, Z. (2025). What and How does In-Context Learning Learn? Bayesian Model Averaging, Parameterization, and Generalization. In *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, pages 1684–1692. PMLR.

Zheng, T., Deng, Z., Tsang, H. T., Wang, W., Bai, J., Wang, Z., and Song, Y. (2025). From Automation to Autonomy: A Survey on Large Language Models in Scientific Discovery.

A Appendix

A.1 Related work

Our framing connects automated science to cognitive accounts of scientific inference. Recent AI-for-science systems increasingly automate parts of discovery, but the harder problem is not only solving a researcher-specified objective; it is proposing, evaluating, and revising the explanatory structures that define the problem itself (Battleday and Gershman, 2024; Cornelio et al., 2023; Novikov et al., 2025). This is also the bridge from Popper-style hypothesis testing to modern AI4Science: discovery requires systems that can treat hypotheses as revisable objects, not just produce successful outputs on a fixed task (Popper, 2005; Lakatos, 2014). Cognitive science offers a useful abstraction for this problem: human learners infer latent structure from sparse data using structured priors and update those hypotheses as examples accumulate (Tenenbaum et al., 2011; Lake et al., 2015).

The number game was introduced as a compact demonstration of how Bayesian hypothesis learning can reconcile rule-based and similarity-based generalization (Tenenbaum, 1999a,b). Later work expanded the empirical dataset and studied richer priors in the same domain (Bigelow and Piantadosi, 2016b,a). This paper uses that tradition as a controlled diagnostic for LLMs rather than as a new cognitive model of human numerical hypotheses.

This perspective also connects our task to in-context learning: the model receives examples in the prompt and must change its beliefs without parameter updates (Brown et al., 2020). Prior analyses have linked in-context learning to online gradient-based optimization (Ahn et al., 2023; Cheng et al., 2024; von Oswald et al., 2023a,b), Bayesian inference and Bayesian model averaging (Müller et al., 2021; Reuter et al., 2025; Xie et al., 2022; Zhang et al., 2025), and related statistical procedures (Bai et al., 2023; Chen and Wang, 2022; Dherin et al., 2025; Garg et al., 2022). Much of this work asks what inference algorithms transformers can implement, often using small models trained from scratch on controlled synthetic task families. We study a complementary setting: pretrained LLMs prompted with a classic hypothesis-learning task, where the context does not merely specify an input–output mapping but incrementally constrains a posterior over possible hypotheses.

The Bayesian process in this setting is explicit. A learner begins with a prior over hypotheses, such as rule-like and interval-like concepts in the number game. Each new example changes the likelihood of those hypotheses: compact hypotheses that contain all examples become more diagnostic than broad compatible hypotheses under strong sampling, whereas weak sampling gives less advantage to narrow hypotheses. The posterior after each example is therefore a reweighted distribution over hypotheses, and predictions are obtained by averaging over that posterior. We use this process not as a claim about the model's internal mechanism, but as a reference algorithm for measuring LLM behavior. In a single measurement and fixed integer domain, pretrained LLMs often look Bayesian-like: their predictions can be fit by a two-parameter Bayesian family that separately tracks prior reliance and likelihood strength, and different examples shift mass toward different rule-like or interval-like hypotheses. The central question is whether this apparent Bayesian updating is the readout of one stable posterior. Our results show that it is only partly so: the same models can show Bayesian-like behavior in one measurement while violating posterior coherence across hypothesis evaluation, hypothesis generation, posterior prediction, and larger-domain generalization.

Several recent studies have tested LLMs on probabilistic reasoning, suspicious coincidence, and number-game-like generalization (Padmanabhan et al., 2025; Qiu et al., 2026; Bazigaran and Sohn, 2025). Our distinguishing contribution is to separate posterior prediction, hypothesis evaluation, and hypothesis generation, while fitting posterior behavior against the same hypothesis space used by the Bayesian reference. This methodological link makes it possible to distinguish a likelihood-level mismatch from the structural evaluation–generation dissociation that would be invisible if any single measurement were treated as sufficient evidence for Bayesian hypothesis learning.

A.2 Model aliases and prompt conditions

The model names in the main article are short aliases used for readability; the experiments are keyed by the corresponding provider or checkpoint identifiers in the engineering configuration. Gemma 4 A4B denotes `google/gemma-4-26B-A4B-it`, Gemma 4 E4B denotes `google/gemma-4-E4B-it`, and Gemma 4 E2B denotes `google/gemma-4-E2B-it` (Team, 2026). Qwen 3.6 A3B denotes `Qwen/Qwen3.6-35B-A3B`, Qwen 3.5 4B denotes `Qwen/Qwen3.5-4B`, and Qwen 3.5 2B denotes

`Qwen/Qwen3.5-2B` (Qwen, 2026). GPT-5.4 Mini denotes `gpt-5.4-mini`. Nemotron 3 Nano denotes `nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16` (NVIDIA et al., 2025). The main non-thinking panel contains all eight aliases. Thinking-mode comparisons use matched thinking and non-thinking runs for Gemma 4 A4B, Gemma 4 E4B, Qwen 3.6 A3B, Qwen 3.5 4B, GPT-5.4 Mini, and Nemotron 3 Nano; Gemma 4 E2B and Qwen 3.5 2B are excluded from those matched comparisons. Posterior prediction is evaluated under the four prompt conditions used in Results: Default Prompt, Strong Prompt, Weak Prompt, and Explicit Prompt. The Default Prompt gives no sampling story; the Strong Prompt states that examples are uniformly drawn from the accepted numbers; the Weak Prompt states only that the examples are positives; the Explicit Prompt supplies the compact candidate-hypothesis list to the prediction measurement.

A.3 Posterior-prediction elicitation

The posterior-prediction quantity $q_m^{(d)}(y \mid X)$ is elicited as a forced binary prediction for each target integer, not as a free-form verbalized probability. For a fixed observed-example set X and query domain D_d, the model first receives the domain, the prompt-condition text, any candidate-list frame required by the condition, and the accepted numbers. It is then queried separately for every $y \in D_d$ with the target question "Would the program say yes to the number y?" and is constrained to answer Yes or No. When the backend exposes answer probabilities, the reported value is the probability assigned to Yes after renormalizing over the Yes and No answer alternatives. For direct model runs this is computed from the first answer-token scores; for hosted or local API-style runs it is computed from the returned answer-token log-probabilities. Thus $q_m^{(d)}(y \mid X)$ is a structured yes/no probability curve over target integers, rather than a distribution the model is asked to write down explicitly.

For thinking-mode posterior prediction, the default protocol separates concept inference from target scoring. The model first receives the same observed-example context and is asked to infer the concept without answering any target question; the resulting thinking state is then used while scoring the Yes/No alternatives for each target. This keeps the target-level readout comparable to the non-thinking log-probability readout, while allowing thinking models to spend their reasoning budget once per observed-example set rather than independently for every target.

As an alternative-elicitation sensitivity check, the same posterior-prediction prompts can be evaluated with repeated categorical answers instead of answer-token probabilities. In this text-response variant, each target question is sampled 20 times at temperature 0.7, and $q_m^{(d)}(y \mid X)$ is estimated as the fraction of valid Yes answers among valid Yes/No responses. This check asks whether a qualitative result depends on reading the model's graded preference from answer-token probabilities rather than from repeated verbal choices. Because the repeated-response estimate is noisier and depends on the sampling temperature, the main analyses use the answer-probability readout whenever it is available and reserve the text-response variant for elicitation sensitivity and for backends where answer probabilities are unavailable.

A.4 Stimulus construction and sequential presentation

The experiment treats a stimulus set as the full set of unique numbers for one trial. These numbers are the possible observed examples in that trial, and the model sees them one at a time. Thus a four-number stimulus set such as $\{16, 8, 2, 64\}$ produces four observed-example presentations: after $\{16\}$, after $\{16, 8\}$, after $\{16, 8, 2\}$, and after $\{16, 8, 2, 64\}$. A one-number stimulus set contributes one presentation. TENENBAUM99 contains eight stimulus sets: two singletons, three rule-like sets, and three similarity-like sets. Sequential presentation expands these into 26 observed-example presentations per domain: 2 singleton, 12 rule-like, and 12 similarity-like presentations. BIGELOW16 contains 255 stimulus sets. Applying the same four-example limit gives 636 presentations in $d = 100$: 55 singleton, 289 rule-like, 14 similarity-like, and 278 other presentations under the structural classifier used in the analysis.

A.5 Candidate hypotheses

For a task t, domain size d, and observed examples X, let $\mathcal{R}_{t,d}$ denote the fixed rule labels available for that task/domain and let $\mathcal{I}_d^{\mathrm{nat}}$ denote the natural interval hypotheses in $\{1, \ldots, d\}$, with endpoints on a 5-integer grid. The Bayesian reference uses the full configured rule-and-interval hypothesis space

$$\mathcal{H}_{t,d} = \mathcal{R}_{t,d} \cup \mathcal{I}_d^{\mathrm{nat}}.$$

For TENENBAUM99, the rule registry uses the identity transform over the full base family: parity, squares, cubes, primes, multiples of 3 through 12, powers of 2 through 10, last-digit rules, and the four $5n + k$ residue classes. Deduplication by extension and removal of very small supports leave 31 rule hypotheses in $d = 100$ and 32 in $d = 200$, for 261 and 892 Bayesian hypotheses after adding the configured natural intervals, respectively. For BIGELOW16, the registry starts from the primordial families in the Bigelow and Piantadosi setting: parity, squares, cubes, primes, and multiples of 3 through 12. It then applies the configured transformations $n + 1$, $n - 1$, $n + 2$, $n - 2$, $2n$, $3n$, $2n + 1$, $3n - 1$, $3n + 1$, 2^n, 2^{n+1}, $2^n + 1$, and $2^n - 1$; after support-based deduplication this gives 128 rule hypotheses in $d = 100$, which are grouped back to 15 base-family labels for prompting. With natural intervals, the BIGELOW16 $d = 100$ Bayesian space contains 358 hypotheses. The configured Bayesian prior $p(h)$ assigns $\lambda = 0.6667$ of its mass uniformly across rule hypotheses and the remaining mass to natural intervals, whose size prior uses an Erlang scale $\sigma = 10.0$; this is the prior used in the Bayesian posterior and the (α, β) family.

The explicit hypothesis-evaluation prompt is not a separate ad hoc list. It exposes a compact, example-conditioned view $K(X)$ of the same rule registry and interval family,

$$K(X) = \mathcal{R}_{t,d}^{\mathrm{prompt}} \cup \{I_{10}(X), I_5(X), I_{\min\max}(X), I_{\mathrm{all}}\} \cup \{\mathrm{other}\}.$$

Here $\mathcal{R}_{t,d}^{\mathrm{prompt}}$ is the task/domain rule part of the same hypothesis registry: 31 rule labels for TENENBAUM99 in $d = 100$, 32 for TENENBAUM99 in $d = 200$, and 15 grouped rule-family labels for BIGELOW16 in $d = 100$. The four interval candidates are also interval-family hypotheses: $I_{10}(X)$ and $I_5(X)$ round the minimum and maximum observed examples outward to multiples of 10 and 5, respectively; $I_{\min\max}(X)$ is the exact interval from the smallest to largest observed example; and $I_{\mathrm{all}} = \{1, \ldots, d\}$. The residual "other" option represents probability assigned to hypotheses not explicitly named by these compact labels. Thus the prompt-visible list has 36 entries for TENENBAUM99 in $d = 100$, 37 entries for TENENBAUM99 in $d = 200$, and 20 entries for BIGELOW16 in $d = 100$, although repeated interval labels can reduce the number of unique displayed labels for some example presentations. This construction gives the model a compact candidate list anchored to the Bayesian hypothesis-space construction without turning hypothesis generation into a list-selection task.

A.6 Alpha–beta fitting objective

In this appendix, X denotes a generic observed-example set, while X_s denotes the observed-example presentation indexed by stimulus set s inside the fitting sum. For model m, observed examples X_s from stimulus set s, and domain target y, let $q_m^{(d)}(y \mid X_s)$ be the model's posterior-prediction readout from the elicitation protocol in Appendix A.3. For any (α, β), the parameterized Bayesian prediction is

$$\hat{q}_{\alpha,\beta}(y \mid X_s) = \sum_{h \in \mathcal{H}} \mathbf{1}[y \in h] \, \frac{\mathbf{1}[X_s \subseteq h] \, p(h)^{\alpha} |h|^{-\beta |X_s|}}{\sum_{h' \in \mathcal{H}} \mathbf{1}[X_s \subseteq h'] \, p(h')^{\alpha} |h'|^{-\beta |X_s|}}.$$

We fit one pair per model/domain/prompt cell and fit scope by minimizing

$$\mathcal{L}(\alpha, \beta) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \frac{1}{|\mathcal{Y}_s|} \sum_{y \in \mathcal{Y}_s} \left(q_m^{(d)}(y \mid X_s) - \hat{q}_{\alpha,\beta}(y \mid X_s) \right)^2,$$

where $\mathcal{Y}_s$ is the set of valid targets for that presentation. In aggregate rows, $\mathcal{S}$ pools task sources before fitting: for a fixed model, domain, hypothesis-space framing, sampling frame, and fit scope, all available TENENBAUM99 and BIGELOW16 stimuli are jointly fit to one (α, β) pair; when only one task source exists for a condition, $\mathcal{S}$ contains that source alone. The "full" scope uses each stimulus set with all configured examples, whereas the $n = 1, 2, 3, 4$ scopes truncate every eligible stimulus set to its first n examples before pooling. The optimizer uses positive parameters, so fitted values are interpreted relative to the Bayesian point $(1, 1)$ rather than as signed effects.

A.7 Cross-measurement projection metrics

All three measurements use the same observed examples $X \subseteq D_d$ and hidden hypothesis $h^\star \subseteq D_d$. Posterior prediction records $q_m^{(d)}(y \mid X)$ for each queried integer $y \in D_d$ using the forced Yes/No protocol described above. Hypothesis evaluation shows the example-conditioned list $K(X)$ defined in Appendix A.5 and records weights over that list. Hypothesis generation does not show $K(X)$; it asks the model to propose 10 candidate hypotheses with corresponding confidences. We also tested longer generation lists in pilot runs. Longer lists mainly added low-confidence tail hypotheses and did not materially change the high-confidence hypotheses on which the analyses depend, so the reported experiments use 10 as a fixed generation budget.

Before projection, hypothesis-evaluation weights and generation confidences are rescaled separately within each returned set so the retained positive weights sum to one. For generation, labels mapped to the same executable rule or interval are collapsed before this rescaling by keeping the larger confidence for that support. This makes the evaluation and generation projections comparable as weighted distributions over matched hypotheses while still reporting unmatched free-text mass separately.

To compare the measurements, we map weighted hypothesis labels into predictive probability functions over D_d. Let $r \in \{\text{eval}, \text{gen}\}$ denote the hypothesis-evaluation or hypothesis-generation measurement, let $L_r(X)$ be the weighted label set returned by measurement r, let $w_r(\ell \mid X)$ be the normalized weight assigned to label ℓ given examples X, and let $S(\ell) \subseteq D_d$ be the support of label ℓ when it can be matched to a rule or interval. For evaluation, $L_{\text{eval}}(X)$ is a weighted subset of the displayed candidate list $K(X)$; for generation, $L_{\text{gen}}(X)$ is the weighted set of model-proposed labels. The projected prediction is

$$\tilde{q}_r(y \mid X) = \sum_{\ell \in L_r(X)} w_r(\ell \mid X)\, \mathbf{1}[y \in S(\ell)].$$

Labels without executable support are reported as unmatched mass and excluded from the projected curve, because projecting them would require adding an external judge.

We measure projected cross-measurement divergence with Jensen–Shannon distance (JSD; base 2) between the normalized projected prediction $\tilde{q}_r(\cdot \mid X)$ and the matched posterior-prediction vector $q_m^{(d)}(\cdot \mid X)$. This bounded symmetric distance is reported separately for hypothesis evaluation and hypothesis generation against posterior prediction.

For top-hypothesis summaries, let $(\ell^\star, w^\star)$ be the top-weighted label and let $S(\ell^\star) \subseteq \{1, \ldots, d\}$ be its executable support when the label can be matched to a rule or interval. We define

$$\text{SupportFrac} = \frac{|S(\ell^\star)|}{d}, \qquad \text{ExampleCons} = \mathbf{1}[X \subseteq S(\ell^\star)].$$

The sum-scaled top-1 confidence is $w^\star$. The top-1 rule indicator is one when $\ell^\star$ maps to a mathematical-rule support and zero otherwise. These metrics ignore unmatched free-text labels except when reporting matched mass or unmatched mass.

A.8 Larger-domain extrapolation metrics

For the larger-domain extrapolation analysis, the observed examples are unchanged and remain in $\{1, \ldots, 100\}$. The prompt changes the queried integer domain, so the model reports a posterior over $\{1, \ldots, 200\}$ rather than over $\{1, \ldots, 100\}$. Let $q_m^{(100)}(y \mid X)$ be the posterior measured over $\{1, \ldots, 100\}$ and let $q_m^{(200)}(y \mid X)$ be the matched posterior over $\{1, \ldots, 200\}$ for the same examples. The unseen-window mass is

$$M_{\text{ext}} = \sum_{y=101}^{200} q_m^{(200)}(y \mid X).$$

To isolate whether the original in-domain shape is preserved, we renormalize the $d = 200$ posterior over the original domain,

$$\tilde{q}_m^{(200\to100)}(y \mid X) = \frac{q_m^{(200)}(y \mid X)}{\sum_{z=1}^{100} q_m^{(200)}(z \mid X)}, \qquad y \le 100,$$

and compute $\mathrm{KL}(q_m^{(100)}(\cdot \mid X) \,\|\, \tilde{q}_m^{(200\to100)}(\cdot \mid X))$. For rule-target discrimination, we average $q_m^{(200)}(y \mid X)$ separately over new-domain targets that satisfy the rule implied by the examples and new-domain targets that do not. This separates calibrated extrapolation to rule-consistent targets from broad leakage into the unseen half of the domain.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Introduction state the paper's scope: a controlled number-game evaluation of LLM posterior prediction, hypothesis evaluation, and hypothesis generation against Bayesian and human references. The claims are tied to the five Results figures, and the Discussion and limitations section explicitly bounds the conclusions to this task family and fixed-cache analysis.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The Discussion and limitations section states the main limitations, including the fixed-cache single-seed design, the narrow task family, the restricted Bigelow16 domain coverage, the prompt mismatch between evaluation and generation, and the low-dimensional nature of the (α, β) fit.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper defines a Bayesian reference model and fitting objectives, but it does not claim new theoretical results requiring formal proofs.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The Methods and Appendix define the task sources, domains, prefix protocol, three posterior measurements, prompt conditions, Bayesian fit, cross-measurement projection metrics, and domain-extension diagnostics. The associated experiment repository documents the default runner, configuration files, cache layout, and analysis outputs needed to reproduce the reported figures.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We provide open access to the experiment code, data-processing scripts, cached outputs needed for analysis, and figure-generation pipeline in the associated repository, with instructions for reproducing the main results.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The Methods and Appendix specify the number-game sources, domains, example-prefix protocol, three measurement interfaces, prompt conditions, candidate-hypothesis construction, fitted metrics, and domain-extension metrics. The study evaluates pretrained models rather than training new models, so optimizer and training hyperparameters are not applicable.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

Justification: The aggregate figures report 95% confidence intervals wherever model-level summaries are averaged. We do not conduct separate statistical significance tests, and the Discussion and limitations section states that the error bars reflect variation across model-level rows rather than repeated stochastic runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: A subset of local model runs used H100 80GB GPUs; local non-thinking runs typically required 6–12 hours per model, and local thinking runs typically required 24–36 hours per model. Most hosted-model experiments used the Tinker API, with additional OpenAI API runs; total compute is approximately proportional to the number of model-condition runs, while hosted API runs report API usage rather than provider-side worker details.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: To the best of our knowledge, the work complies with the NeurIPS Code of Ethics: it analyzes existing pretrained language models on synthetic and previously published number-game stimuli, without collecting private data or deploying a system that affects users.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Introduction and Discussion motivate the positive impact of better diagnostics for scientific and agentic LLM use, while the Results and Discussion emphasize the negative implication that fluent hypothesis generation can mask incoherent posterior use. The work is foundational and does not introduce a deployed system.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a new pretrained model, high-risk scraped dataset, or other asset requiring controlled access. It analyzes existing language models on synthetic and previously published number-game stimuli.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: We credit the original number-game sources and use their public releases under the stated terms: the Tenenbaum-style human-rating CSVs are from the GPLv3-licensed `humanlike_fewshot_learning` GitHub repository, and the Bigelow16 number-game dataset is released publicly under Creative Commons Attribution 4.0 (CC BY 4.0). Pretrained model assets are used under the licenses and provider terms listed by their original GitHub, Hugging Face, or API-provider pages.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We release the experiment code, cached analysis artifacts, and figure-generation pipeline with repository documentation. We do not release a new pretrained model or a newly collected human-subject dataset.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not collect new human-subject data or use crowdsourcing. Human behavior appears only as previously published aggregate baselines from the cited number-game literature.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: No new human-subject experiments, user studies, or crowdsourcing are conducted for this paper, so IRB approval or equivalent review is not applicable for the present study.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: LLMs are the core experimental objects of study. The Methods and Appendix describe how pretrained LLMs are prompted and evaluated in posterior prediction, hypothesis evaluation, and hypothesis generation modes, including prompt-condition and thinking comparisons.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.